

CS & IT ENGINEERING



Operating System

Memory Management

Lecture -1

By- Vishvadeep Gothi sir



Recap of Previous Lecture



doubt ?

Topic

Banker's Algorithm

Topic

Deadlock Detection



Topics to be Covered



Topic

Memory Management

Topic

**Contiguous Memory Management
Technique**



Topic : Banker's Algorithm

Process	Allocation	Max	Available	Need
	A B C	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2	7 4 3
P ₁	2 0 0	3 2 2		1 2 2
P ₂	3 0 2	9 0 2		6 0 0
P ₃	2 1 1	2 2 2		0 1 1
P ₄	0 0 2	4 3 3		4 3 1

#Q. What will happen if process P0 requests one additional instance of resource type C ?

What will happen if process P3 requests one additional instance of resource type B and one instance of resource type C?

Both requests granted



Solution 1

no deadlock (livelock)

Boolean lock=false;

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

Solution 2

no deadlock

```
int turn=0;
```

```
while(true)
{
    while(turn!=0);
    CS
    turn=1;
    RS;
}
```

```
while(true)
{
    while(turn!=1);
    CS
    turn=0;
    RS;
}
```

Peterson's Solution

```
Boolean Flag[2]={False, False};
int turn;
```

```
while(true) {
    Flag[0]=true;
    turn=1;
    while(Flag[1] && turn==1);
        CS
    Flag[0]=False;
    RS;
}
```

no deadlock

```
while(true){
    Flag[1]=true;
    turn=0;
    while(Flag[0] && turn==0);
        CS
    Flag[1]=False;
    RS;
}
```

Question 1

```
turn=0;
```

```
while(true)
{
    while(turn);
    turn=1;
    //CS
    turn=0;
    RS;
}
```

no deadlock

```
while(true)
{
    while(turn);
    turnlock=1;
    //CS
    turnlock=0;
    RS;
}
```

Question 2

lock=False;

```
while(true)  
{  
    while(lock!=False);  
    CS  
    lock=True;  
    RS;  
}
```

no deadlock

```
while(true)  
{  
    while(lock!=True);  
    CS  
    lock=False;  
    RS;  
}
```

Question 3

`lock=False;`

`while(true)` *P1, P2*
{
 `while(lock ==False)`
 {
 `lock = True;`
 }
 CS
 `lock=False;`
 RS;
}

no deadlock

Question 4

Boolean lock= True;

```
while(true)  
{  
    while(lock)  
    {  
        CS  
        lock = False  
    }  
    lock=True;  
    RS;  
}
```

no deadlock



Topic : TestAndSet()



no deadlock

```
Boolean Lock=False;
```

```
boolean TestAndSet(Boolean *trg){
```

```
boolean rv = *trg;
```

```
*trg = True;
```

```
Return rv;
```

```
}
```

```
while(true)
```

```
{
```

```
while(TestAndSet(&Lock));
```

```
CS
```

```
Lock=False;
```

```
}
```



Topic : Swap()



Boolean Key; //Local

Boolean Lock=False; //Shared

void Swap(Boolean *a, Boolean *b)

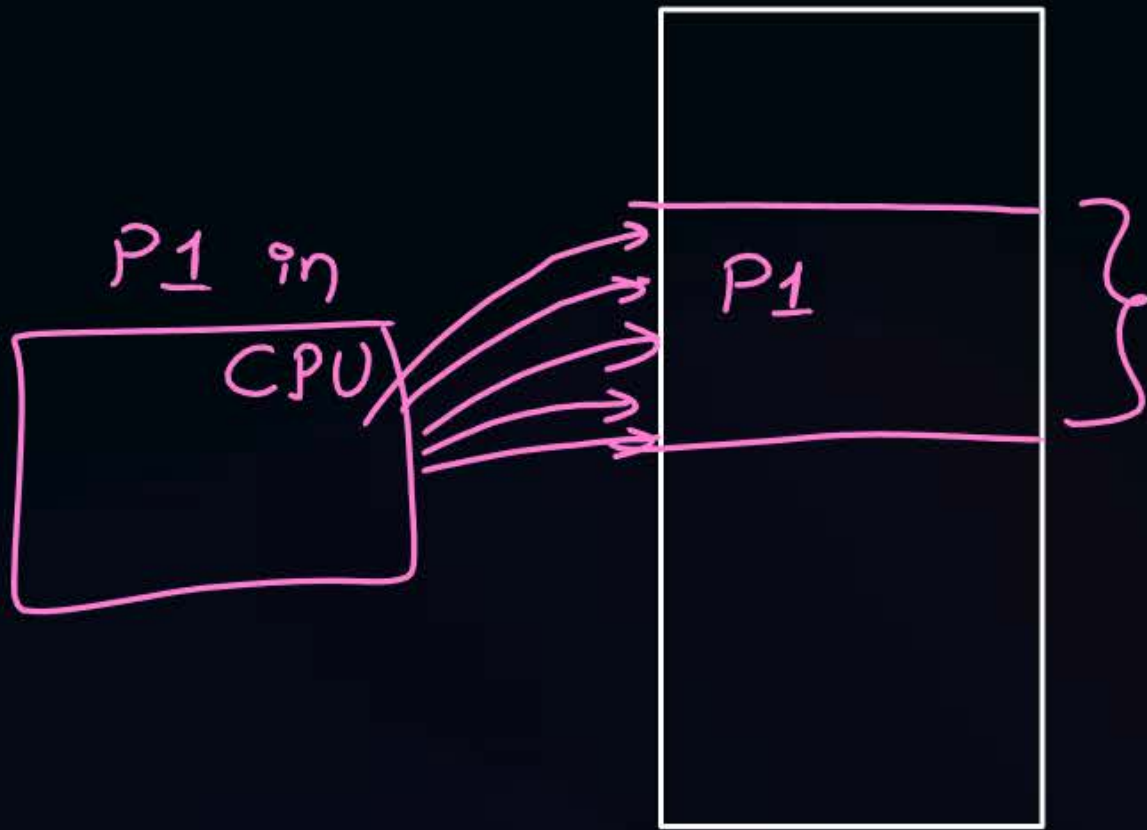
```
{  
    boolean temp = *a;  
    *a=*b;  
    *b=temp;  
}
```

no deadlock

```
while(true){  
    Key = True;  
    while (key==True)  
        Swap(&Lock, &Key);  
    CS  
    Lock=False;  
}
```



Topic : Memory Management





Topic : Functions of Memory Management



1. Memory allocation
2. Memory deallocation
3. Memory protection $\Rightarrow$ a process can access only its own content from memory.



Topic : Goals of Memory Management

1. Maximum Utilization of space
2. Ability to run larger programs with limited space $\Rightarrow$ using virtual memory



Topic : Memory Management Techniques

Contiguous

Entire process should be stored on consecutive memory locations



fixed partition
↓
suffers from internal fragmentation

variable partition
↓
suffers from external fragmentation

Non-Contiguous

Entire process divided into partitions and each partition is stored on different mem. locations



Paging*

segmentation



Topic : Contiguous Memory Management

- Entire process should be stored on consecutive memory locations



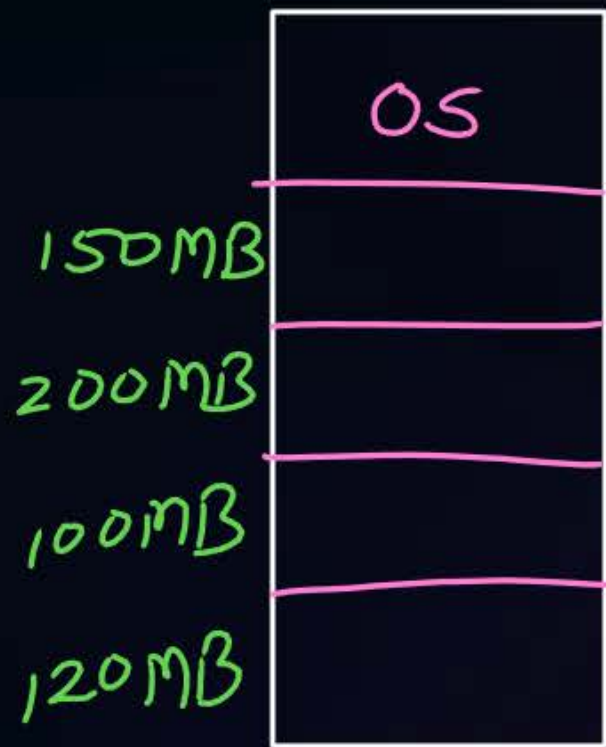
Topic : Fixed Partition Contiguous MMT

mem. is divided into fixed number of partitions, and each partition can be used to accommodate one process.

→ max. degree of multiprogramming is limited by number of partitions.

→ It suffers from internal fragmentation.

↓
if extra space given to process which is more than required space then that extra wasted space is internal fragmentation.





Topic : Partition Allocation Policy

which partition should be allocated to a new process.

1. First fit $\Rightarrow$ first partition from starting which can store process
2. Best fit $\Rightarrow$ smallest possible partition —||—
3. worst fit $\Rightarrow$ Biggest —||—
4. next fit $\Rightarrow$ first partition from previously allocated partition



Topic : Partition Allocation Policy

4 partitions of size: 100KB, 120KB, 150KB and 80KB

Allocation

P1 (Size = 110KB)

P2 (Size = 70KB)

First Fit

120KB

100KB

Best Fit

120KB

80KB

Worst Fit

150KB

120KB

Next Fit

120KB

150KB

Total internal fragment

$$10 + 30 = 40 \text{KB}$$

$$10 + 10 = 20 \text{KB}$$

$$40 + 50 = 90 \text{KB}$$

$$10 + 80 = 90 \text{KB}$$

→ Best fit provide min. internal fragmentatⁿ but takes time to find the smallest partition

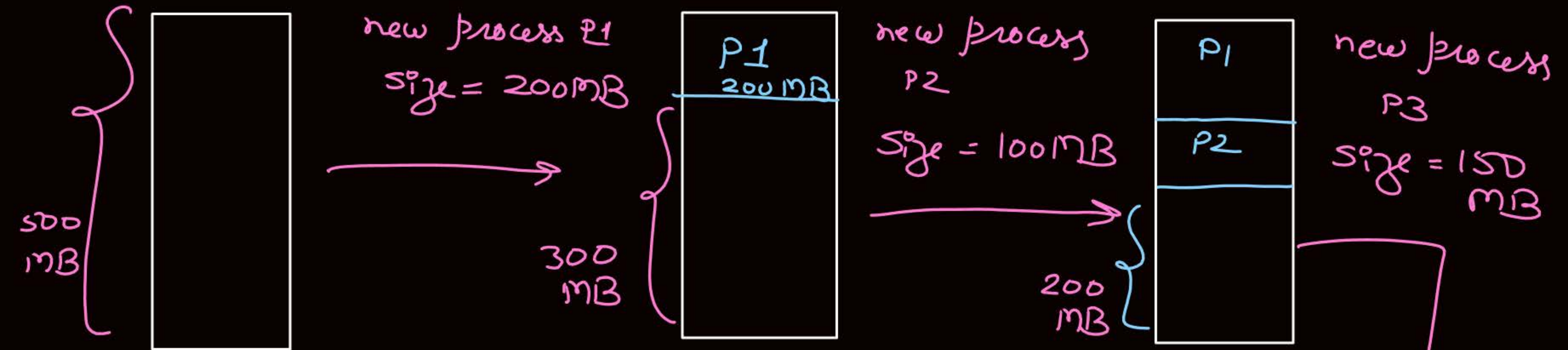
→ first fit and next fit are fastest.



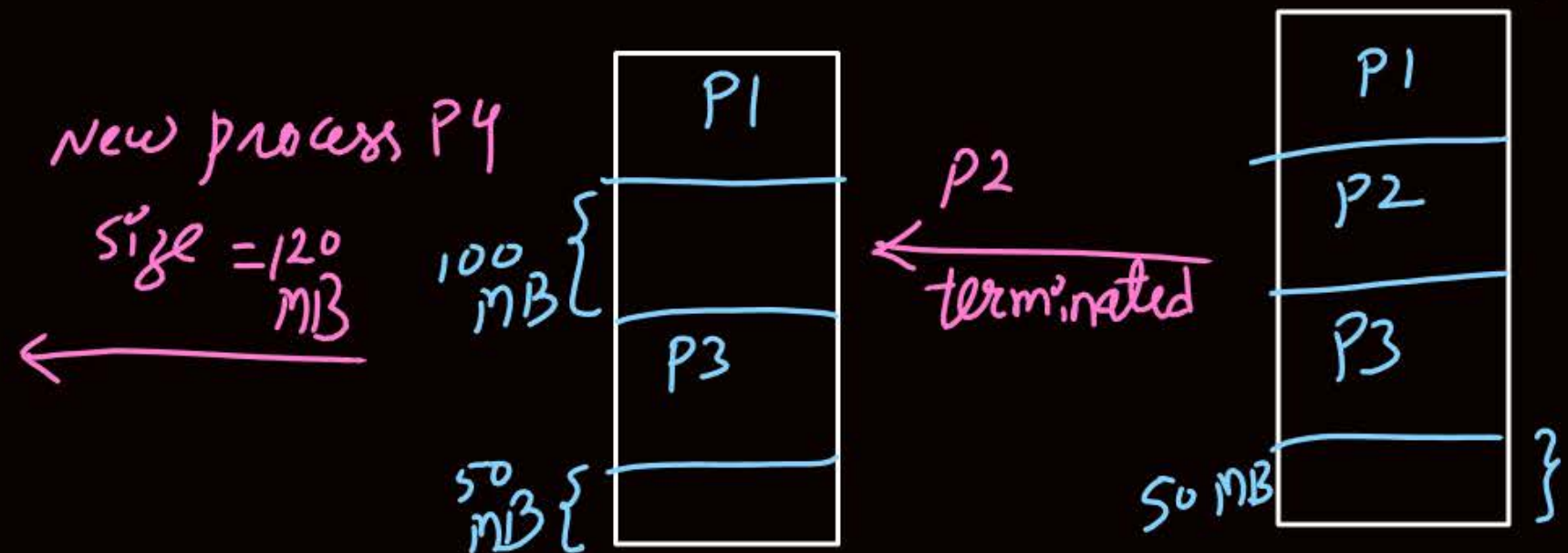
Topic : Variable Partition Contiguous MMT

whenever a new process arrives, a new partition is created of the size exactly equal to the size of process, to store the process.

no any internal $\downarrow$ fragmentation.



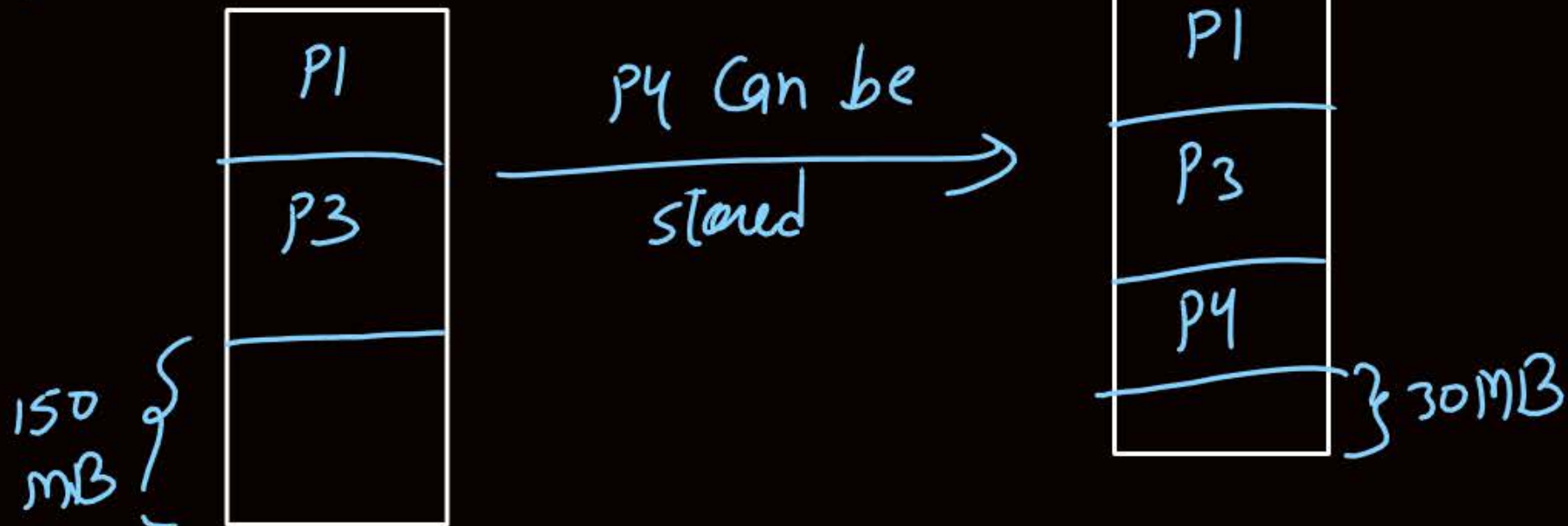
external fragmentation
The process P4
can not be stored in
main mem. as 120MB
consecutive space is
not available.



solution of External Fragmentation $\Rightarrow$ Compaction

Collect all allocated processes into one side of memory so that other side of memory has all empty space together to store new process.

After Compaction



[MCQ]



#Q. Consider the requests from processes in given order ~~300K~~, ~~25K~~, ~~125K~~, and 50K. Let there be two blocks of memory available of size 150K followed by a block size 350K. Which of the following partition allocation schemes can satisfy the above requests? (*variable partition*)

holes

- A** Best fit but not first fit
- B** ✓ First fit but not best fit
- C** Both First fit & Best fit
- D** neither first fit nor best fit





2 mins Summary

Topic

Memory Management

Topic

**Contiguous Memory Management
Technique**





Happy Learning

THANK - YOU

